

Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents must integrate external tool returns into ongoing reasoning—a capability that standard left-to-right pretraining on code exposes only in its forward
2 direction. We observe that the *action* $\rightarrow$ *observation* $\rightarrow$ *continuation* loop of
3 a coding agent is structurally isomorphic to a function call site, where a caller
4 binds arguments, a callee returns a value computed elsewhere, and downstream
5 code consumes that value. This conditioning structure exists at internet scale
6 in ordinary code. We exploit it through *function-aware fill-in-the-middle (FIM)*
7 *mid-training*: a self-supervised objective that masks functions selected via program
8 dependency graph analysis and a complexity–inferability double criterion. We
9 mid-train Qwen2.5-Coder-Instruct (7B/14B) and Qwen3-8B on a 2.6B-token
10 decontaminated corpus drawn from 968 GitHub repositories, then apply existing
11 agentic post-training pipelines. Mid-training improves SWE-Bench-Verified
12 by +2.8/ + 3.0 at 7B/14B and by +3.2 on Qwen3-8B; SWE-Bench-Lite gains
13 are +3.7/ + 4.0/ + 5.4 on the same models. The improvement holds across
14 two post-training pipelines (R2E-Gym, SWE-Smith) and on a non-Qwen2.5 base
15 (Qwen3-8B with SWE-Lego). Beyond in-domain gains, mid-training also mitigates
16 the capability erosion that agentic post-training otherwise inflicts on non-agent coding
17 (e.g., LiveCodeBench) and non-coding tool-use benchmarks (τ -bench, BFCL):
18 although the mid-training corpus contains Python code only, the function-call
19 inductive bias survives post-training and yields consistent gains.
20

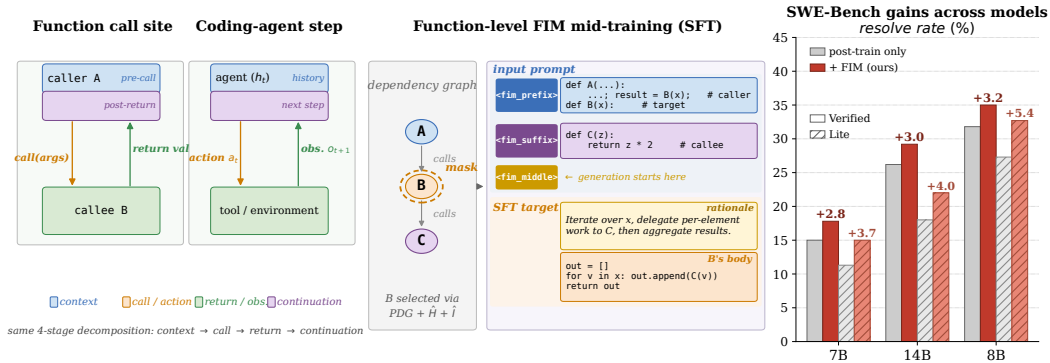


Figure 1: **Left:** A function call site and a single step of a coding agent are *structurally similar*, decomposing into the same four stages: context, call/action, return/observation, continuation. **Middle:** We exploit this analogy via function-aware FIM mid-training. A function B is selected from the program dependency graph using complexity ($\hat{H}$) and inferability ($\hat{I}$) scores; the model is then mid-trained to fill in B 's body together with a chain-of-thought rationale, given the surrounding file as an FIM-formatted prompt. **Right:** Mid-training yields consistent gains across both Qwen2.5-Coder (7B, 14B) and Qwen3 (8B) on SWE-Bench-Verified (solid bars) and SWE-Bench-Lite (hatched bars).

1 Introduction

Coding agents that resolve real software engineering issues have moved from research demos to deployed systems [10, 22, 19]. Their progress, however, has been driven almost entirely by scaling synthetic agent-trajectory data during post-training: pipelines such as SWE-Gym [14], R2E-Gym [9], and SWE-Smith [23] curate or synthesize trajectories that imitate human or LLM behavior on issue-resolution tasks. The base model these pipelines start from is typically a code LLM trained with next-token prediction (and, in some cases, random-span FIM) on internet-scale code [7, 5, 11]. Between these two stages lies a training-time gap: the base model is rarely optimized for the conditioning structure that agentic post-training will later demand. We treat this gap as an opportunity for a dedicated *mid-training* stage that aligns the base model with agent-relevant inductive biases before agent-specific data is introduced.

Our central observation is that the inductive bias required by a coding agent already exists in ordinary code, but in a shape that left-to-right pretraining systematically under-exposes. At each step an agent maintains a history h_t , samples an action $a_t \sim \pi(a_t | h_t)$, receives an observation o_{t+1} produced by an external process, and continues conditioned on the entire trace. This four-part decomposition—context, action, externally-computed return, continuation—is precisely the decomposition of a function call site: pre-call code that establishes intent and binds arguments; the call itself; a return value produced by code outside the immediate scope; and downstream code that consumes the return value (Figure 1, left). A model trained to reason *bidirectionally* about function-level dependencies must learn to reconstruct a callee’s behavior from caller context and downstream usage, which is the same competence required to predict an agent’s continuation given a history and a tool return. The correspondence is structural rather than literal—FIM training conditions on a given suffix while agent inference generates one—but it suggests that representations induced by the former should transfer to the latter, an empirical question we address in Section 3.

The fill-in-the-middle objective is not new [1]; recent code LLMs mix random-span FIM into pretraining [7, 5, 11]. Random-span FIM is nonetheless poorly aligned with agentic conditioning for three reasons. (i) *Span boundaries are syntactically arbitrary*: most masked spans cut through expressions or partial statements and carry weak signal about function-level dependencies. (ii) *There is no reasoning supervision*: the model fills the span directly, with no intermediate rationale mirroring an agent’s think-then-act pattern. (iii) *The objective is dissolved into pretraining*: by the time post-training begins, any structural prior conferred by FIM has been amortized across trillions of unrelated tokens. We address all three points: masking targets are selected at function granularity using program dependency graph analysis with a complexity–inferability double criterion that is deliberately base-model-agnostic (Section 2.3); chain-of-thought rationales are embedded *inside* the FIM middle span so the model produces reasoning consistent with the eventual code (Section 2.4); and the objective is applied at a dedicated mid-training stage, concentrating its signal immediately before agentic post-training.

We evaluate this recipe along three axes of robustness. *On the Qwen2.5-Coder-Instruct series*, mid-training improves SWE-Bench-Verified by +2.8 and +3.0 points at 7B and 14B respectively, indicating that the structural prior is not absorbed by larger pretrained models in the practical deployment range. *Across two post-training pipelines*, mid-training improves both R2E-Gym and SWE-Smith on the same 7B base, with the SWE-Smith pairing yielding +5.3 points on SWE-Bench-Verified. *On a non-Qwen2.5 base*, mid-training transfers to Qwen3-8B (paired with SWE-Lego) for a +3.2-point gain on SWE-Bench-Verified; this single comparison varies the post-training pipeline jointly with the base model and should be read as evidence that the prior is not specific to a single Qwen2.5-Coder + R2E-Gym/SWE-Smith combination, rather than as a guarantee across families.

A second set of results probes our motivating hypothesis. All benchmarks in this paper are evaluated on checkpoints that have undergone the full pipeline—either post-training alone (baseline) or mid-training followed by post-training (ours)—because a mid-trained checkpoint without subsequent instruction-style post-training has degraded instruction-following and is not directly comparable to instruction-tuned baselines. Under this protocol, agentic post-training alone substantially regresses non-target capabilities at 14B, dropping LiveCodeBench, BFCL, and τ -bench by double-digit margins in some cases. Adding mid-training before the same post-training restores +11.1 points on LiveCodeBench, +2.4 on BFCL, and +3.9 on τ -bench. Because the mid-training corpus contains only Python code with no tool-use data, mid-training has no a priori reason to affect τ -bench or

76 BFCL; the fact that it does, in the same direction as the in-domain SWE-Bench gains, provides direct
 77 evidence for the isomorphism between function calls and agent tool calls.

78 **Contributions.** (1) **Conceptual framing.** We articulate a structural correspondence between
 79 function call sites and the action–observation–continuation loop of coding agents, and motivate
 80 function-granularity FIM as a self-supervised prior for agent capability that exists at internet scale in
 81 ordinary code. (2) **Method.** We introduce a function-aware FIM mid-training pipeline combining
 82 program dependency graph analysis, a hand-designed complexity–inferability double criterion that is
 83 base-model-agnostic, and CoT rationales embedded inside the FIM middle span. (3) **Robustness**
 84 **evaluation.** We validate the recipe along three axes—two model sizes within Qwen2.5-Coder-Instruct,
 85 two post-training pipelines, and one alternative base (Qwen3-8B with SWE-Lego)—observing
 86 consistent in-domain gains in every configuration. (4) **Direct test of the motivating hypothesis.**
 87 The same Python-only mid-training corpus—which contains no tool-use data—also yields gains on
 88 tau-bench, BFCL, and LiveCodeBench, providing direct evidence that the function-call inductive
 89 bias survives agentic post-training and transfers across task families. (5) **Open release.** We release
 90 the 968-repository decontaminated corpus (78K filtered self-contained Python files, $\approx$ 33M lines),
 91 the full selection pipeline, and all mid-training checkpoints to support reproduction.

92 2 Method

93 2.1 Motivation: Function Calls as Agent-Like Structures

94 At step t a coding agent maintains a history h_t , samples an action $a_t \sim \pi(a_t \mid h_t)$, receives an
 95 observation $o_{t+1} \sim p(o_{t+1} \mid h_t, a_t)$, and continues conditioned on the trace. Under the function-
 96 call/agent-step correspondence (Figure 1, left), the four parts—history, action, return, continuation—
 97 align respectively with pre-call context, the call, the returned value, and downstream usage. This
 98 isomorphism motivates extracting an agentic conditioning signal directly from source code via a fill-
 99 in-the-middle (FIM) objective. The correspondence is *structural* rather than literal—FIM conditions
 100 on a given suffix while agent inference generates one—but a model that reasons bidirectionally about
 101 function-level dependencies should learn a representation that transfers when the suffix is generated
 102 rather than observed. Standard random-span FIM [7, 5, 11] captures this structure only incidentally;
 103 we construct a *function-aware* variant whose masking targets are chosen by program structure and
 104 contextual predictability.

105 2.2 Data Collection and Decontamination

106 We curate a corpus of 968 Python repositories from GitHub. Starting from $\sim$ 2,000 candidates
 107 retrieved by combining a star-count threshold with ten topic queries, we apply manual quality filtering
 108 and remove every repository whose origin overlaps with the source repositories of SWE-Bench [10]
 109 (verified by repository name and any known fork). To eliminate test-time leakage, we restrict each
 110 repository to commits whose timestamp precedes the earliest base-commit used in SWE-Bench-
 111 Verified and SWE-Bench-Lite. The filtered corpus contains $\approx$ 78K self-contained Python files
 112 (averaging 428 lines per file) and yields $\sim$ 2.6B training tokens under the Qwen2.5-Coder tokenizer,
 113 spanning 320K single-function, 60K pair, and 20K triple FIM samples; full statistics, topic-category
 114 breakdown, and license inventory are reported in Appendix A, and the full repository list will be
 115 released to support reproduction.

116 2.3 Function-Aware FIM Target Selection

117 Given each source file, our pipeline produces a set of mask targets together with the corresponding
 118 masked file. Each target is a single function or a connected group of 2–3 functions; the multi-function
 119 variant is studied in Section 3.4. The pipeline has four stages: dependency-graph construction,
 120 complexity scoring, inferability scoring, and threshold-based selection. Figure 2 illustrates these
 121 stages on a small running example—a Calculator class with two top-level helpers. The full single-
 122 function selection algorithm is given in Appendix B.1, the multi-function extension in Appendix B.7,
 123 and a numerical walkthrough of every quantity shown in Figure 2(b) in Appendix B.6.

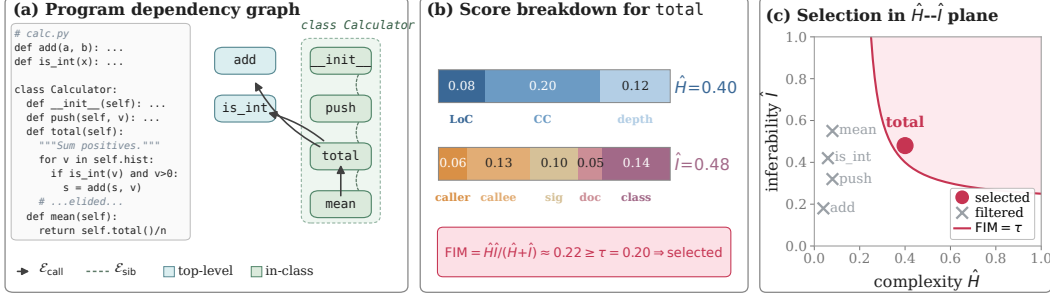


Figure 2: Function-aware FIM target selection on a small calculator example. **(a)** Program dependency graph parsed from the AST: solid arrows are call edges $\mathcal{E}_{\text{call}}$, dashed lines are sibling edges $\mathcal{E}_{\text{sib}}$ between same-class methods. **(b)** Stacked bars decompose the complexity score $\hat{H} = 0.40$ (Eq. 1; LoC, CC, depth) and the inferability score $\hat{I} = 0.48$ (Eq. 2; five context signals) for `Calculator.total`, yielding $\text{FIM} \approx 0.22 \geq \tau$. **(c)** In the $(\hat{H}, \hat{I})$ plane, the four short helpers fall outside the selected region while `Calculator.total` is kept.

2.3.1 Program Dependency Graph

For each file we parse its AST and extract the set $\mathcal{V}$ of function nodes (top-level functions and class methods, identified by qualified names). Two edge sets are constructed: *call edges* $\mathcal{E}_{\text{call}}$ between caller and callee, and *sibling edges* $\mathcal{E}_{\text{sib}}$ between methods of the same class (capturing intra-class coupling that does not manifest as direct calls but carries information through shared instance state). Call resolution handles common Python idioms (direct invocations, class instantiation, `self/ccls` method calls) with a short-name fallback against the qualified-name index; Appendix B.2 gives the full procedure.

2.3.2 Complexity Score $\hat{H}$

For each $v \in \mathcal{V}$ we define

$$\hat{H}(v) = w_\ell \phi(\text{LoC}(v), c_\ell) + w_c \phi(\text{CC}(v), c_c) + w_d \phi(\text{D}(v), c_d), \quad (1)$$

where $\text{LoC}(v)$ is lines of code, $\text{CC}(v)$ is McCabe cyclomatic complexity, $\text{D}(v)$ is the maximum nesting depth of control-flow constructs, and $\phi(x, c) = \min(x/c, 2)$ normalizes each quantity by a soft cap. Caps and weights are listed in Appendix B.3.

2.3.3 Inferability Score $\hat{I}$

A target should be *recoverable* from the surrounding context. $\hat{I}(v)$ aggregates five context-derived signals that approximate the mutual information between v 's body and the rest of the file:

$$\hat{I}(v) = \alpha C_{\text{caller}}(v) + \beta C_{\text{callee}}(v) + \gamma C_{\text{sig}}(v) + \delta C_{\text{doc}}(v) + \varepsilon C_{\text{class}}(v). \quad (2)$$

C_{caller} scores call-site argument specificity, C_{callee} counts intra-file functions called by v , C_{sig} aggregates type annotations and name descriptiveness, C_{doc} indicates docstring presence, and C_{class} counts in-class siblings; full component formulas and weights are in Appendix B.4. Each component is a hand-designed proxy: a learned predictability score would couple selection to a particular reference model and complicate the cross-base-model generalization analysis (Section 3.2).

2.3.4 Single-Function Score

We combine $\hat{H}$ and $\hat{I}$ in a harmonic-mean-like form, scaled by a one-sided difficulty penalty $\rho(\Delta(v))$:

$$\text{FIM}(v) = \frac{\hat{H}(v) \hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v)). \quad (3)$$

The harmonic-mean form forces both $\hat{H}$ and $\hat{I}$ to be large simultaneously, penalizing imbalance; ρ down-weights targets that remain hard even given full context, which would otherwise be unlearnable noise. Hard filters on length and dunder methods, the form of ρ , and the threshold $\tau = 0.20$ used throughout this work are documented in Appendix B.5.

2.3.5 Multi-Function Group Selection

Real-world code patches frequently span multiple related functions [10], motivating an extension that masks *groups* of $k=2$ or $k=3$ structurally connected functions; we use this variant in our main recipe and ablate it in Section 3.4. A group score $\text{FIM}(G)$ multiplies a coupling term, the harmonic-mean-like product of group-level $\hat{H}(G)$ and $\hat{I}(G)$, and a difficulty penalty, with $\hat{I}(G)$ recomputed under joint masking so that intra-group references cannot inflate the score. The eight topology patterns (caller-callee, co-callee, sibling-coupled, mutual-call, call-chain, hub, fan-in, class-triad), the full equations, Algorithm 2, and a worked pair example are in Appendix B.7.

2.4 Chain-of-Thought Augmentation

For each selected target we generate a natural-language rationale via Gemini-3-Flash prompted with the masked file and the original function body (template in Appendix B.9). The rationale is embedded *inside* the FIM middle span, immediately before the function body:

```
<fim_prefix> <prefix> <fim_suffix> <suffix> <fim_middle> <rationale> <body>
```

The model is therefore trained to produce, in order, a reasoning trace followed by code consistent with that trace, mirroring the think-then-act structure of an agent step. To control quality, each (CoT, completion) pair is then scored by Gemini-3-Flash given the masked file, the rationale, the generated completion, and the ground-truth function body; we retain only the top-scoring $\sim 400\text{K}$ samples as our final training corpus. The verification prompt template is in Appendix B.9. Section 3.4 isolates the contribution of FIM structure from CoT distillation via a *self-CoT* variant in which the model under training generates its own rationales.

3 Experiments

3.1 Setup

Benchmarks. We evaluate on three groups. *Coding-agent benchmarks:* SWE-Bench-Verified (500) and SWE-Bench-Lite (300) [10], our primary in-domain target. *Non-agent coding benchmarks:* LiveCodeBench [8], OJBench [21], and FullStackBench-EN [2]; these probe pure code generation and serve as a regression check. *Tool-use and OOD agent benchmarks:* Terminal-Bench [12], τ -bench [24], and BFCL [15]; the latter two contain no Python code-editing trajectories and test whether the function-call inductive bias transfers across task families.

Mid-training and post-training pipeline. We mid-train Qwen2.5-Coder-Instruct (7B/14B) [7] and Qwen3-8B [16] (base, not Instruct) on the selected FIM corpus using the standard FIM loss on the middle span (rationale plus body), packed to the model’s native context length and using its native FIM sentinel tokens. We then apply existing agentic post-training pipelines without modification: R2E-Gym [9] or SWE-Smith [23] for the Qwen2.5-Coder runs, and SWE-Lego [18] for Qwen3-8B. For Qwen3-8B we train SWE-Lego for 2 epochs rather than the official 4 to prevent overfitting in our setup. Hyperparameters and token budgets are in Appendix C.

Evaluation protocol. All numerical results are means over three independent evaluation seeds on the final checkpoint of each pipeline; the “%” symbol is omitted inside table cells, and standard-deviation bands are reported in parentheses where computed over three seeds. “Baseline” refers to the base model followed by an agentic post-training pipeline; “ours” refers to the same base followed by FIM mid-training and then by the identical post-training pipeline. We do not evaluate mid-training-only checkpoints, because FIM data without subsequent instruction-style post-training degrades instruction-following and makes any comparison against instruction-tuned baselines unfair. Every gain we attribute to mid-training is therefore one that *survives* subsequent post-training. The agent harness is fixed by the post-training pipeline (R2E-Gym uses an OpenHands fork [19, 9]; SWE-Smith uses SWE-agent [22]; the Qwen3-8B runs use OpenHands directly because SWE-Lego post-training data exceeds the 32K context of the R2E-Gym/SWE-Smith scaffolds).

Models and baselines. For Qwen2.5-Coder-Instruct at 7B/14B [7] we report (i) the instruction-tuned base, (ii) post-training with R2E-Gym reproduced under our setup, (iii) the official R2E-Gym numbers (in grey, where available), (iv) our recipe of FIM mid-training followed by R2E-Gym post-training, and at 7B additionally the analogous SWE-Smith [23] variants. Qwen3-8B uses the SWE-Lego [18]

Table 1: Main results on coding agent benchmarks. All numbers are percentages, means over three independent seeds with std bands; *Average* is the unweighted mean of Verified and Lite. Bolded rows are our recipe; rows tagged *officially reported* are quoted from the corresponding original publications. SWE-Lego is post-trained for 2 epochs (vs. 4 in the official release) to prevent overfitting.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
<i>Qwen2.5-Coder-7B-Instruct</i>			
— (no agentic training)	1.8 (± 1.3)	1.0 (± 1.0)	1.40
+ R2E-Gym [9] (reproduced)	15.00 (± 1.5)	11.33 (± 1.2)	13.17
+ R2E-Gym (officially reported)	19.00 (± 1.0)	11.00 (± 0.8)	15.00
+ FIM-Midtrain + R2E-Gym	17.80 (± 1.4)	15.00 (± 1.1)	16.40
Δ (ours vs. reproduced)	+2.80	+3.67	+3.24
+ SWE-Smith [23] (reproduced)	12.30 (± 1.2)	14.20 (± 1.4)	13.25
+ SWE-Smith (officially reported)	15.20	11.70	13.45
+ FIM-Midtrain + SWE-Smith	17.60 (± 1.3)	14.70 (± 1.0)	16.15
Δ (ours vs. reproduced)	+5.30	+0.50	+2.90
<i>Qwen2.5-Coder-14B-Instruct</i>			
— (no agentic training)	4.0 (± 1.6)	2.7 (± 1.0)	3.35
+ R2E-Gym (reproduced)	26.20 (± 1.4)	18.00 (± 1.1)	22.10
+ R2E-Gym (officially reported)	26.80 (± 1.4)	20.67 (± 0.7)	23.74
+ FIM-Midtrain + R2E-Gym	29.20 (± 1.5)	22.00 (± 1.2)	25.60
Δ (ours vs. reproduced)	+3.00	+4.00	+3.50
<i>Qwen3-8B</i>			
— (no agentic training)	7.60 (± 1.2)	5.80 (± 0.9)	6.70
+ SWE-Lego [18] (reproduced)	31.80 (± 1.0)	27.30 (± 1.1)	29.55
+ FIM-Midtrain + SWE-Lego	35.00 (± 1.5)	32.70 (± 1.3)	33.85
Δ (ours vs. reproduced)	+3.20	+5.40	+4.30

pipeline; the cross-base-model comparison thus varies the post-training pipeline simultaneously, a confound we discuss in Section 3.2.

3.2 Main Results on SWE Agent Benchmarks

Table 1 summarizes in-domain agent results.

Consistent gains on the Qwen2.5-Coder series. Holding the post-training pipeline fixed at R2E-Gym, FIM mid-training improves SWE-Bench-Verified by +2.80 on 7B-Instruct and +3.00 on 14B-Instruct, with matching directional gains on SWE-Bench-Lite (+3.67/ +4.00). Both Qwen2.5-Coder-Instruct sizes benefit from the same mid-training corpus and recipe, indicating that the structural prior is not absorbed by the larger pretrained checkpoint within this family.

Transfer across post-training pipelines. Replacing R2E-Gym with SWE-Smith on the same 7B base yields +5.30 points on Verified, larger than the +2.80 under R2E-Gym; on Lite the SWE-Smith pairing gains only +0.50, smaller than +3.67 for the R2E-Gym pairing. The two pipelines together indicate that mid-training is not tuned to a single post-training data distribution, though the magnitude of its benefit depends on the baseline pipeline it composes with.

Transfer to a non-Qwen2.5 base. Switching to Qwen3-8B paired with SWE-Lego, mid-training improves Verified by +3.20 and Lite by +5.40, comparable to the Qwen2.5-Coder gains. This single comparison varies the post-training pipeline jointly with the base model (SWE-Lego is required because Qwen3-8B’s 40,960-token context exceeds the R2E-Gym/SWE-Smith scaffolds), so the result should be read as “not specific to the Qwen2.5-Coder + R2E-Gym/SWE-Smith pairing” rather than as a guarantee across base-model families.

3.3 Capability Preservation and Cross-Domain Transfer

A natural concern with task-specialized post-training is that it erodes capabilities the base model already had. We test this on six benchmarks outside SWE-Bench and—for τ -bench and BFCL—

Table 2: Capability preservation and cross-domain transfer at 14B with R2E-Gym. Bold marks the better trained variant per column (post-only vs. ours); the Instruct row is shown as a reference ceiling. All cells are percentages.

Setting	Non-agent coding			Agent OOD	Tool use		Avg
	LiveCode	OJBench	FSB-EN	Terminal	τ -bench	BFCL	
Instruct Model	37.20	5.20	53.80	0.00	5.70	23.20	20.85
+ R2E-Gym	24.10	2.80	47.72	2.41	3.40	15.80	16.04
+ FIM Mid-Train + R2E-Gym	35.20	4.74	48.25	3.66	7.30	18.20	19.56
Δ (vs. R2E-Gym only)	+11.10	+1.94	+0.53	+1.25	+3.90	+2.40	+3.52

outside the coding domain entirely. Because the controlled triple (instruct base vs. post-training only vs. mid-training + post-training) is the most expensive to run, we conduct it at 14B with R2E-Gym (Table 2).

Agentic post-training has a substantial hidden capability cost. R2E-Gym alone reduces every non-agent and tool-use benchmark relative to the Instruct ceiling: LiveCodeBench by 13.10, BFCL by 7.40, FullStackBench-EN by 6.08, τ -bench by 2.30, and OJBench by 2.40. Averaged across the six benchmarks the post-trained model loses 4.81 points relative to Instruct—the implicit cost paid for SWE-Bench gains, rarely highlighted in agent papers.

Mid-training largely closes the regression gap. Adding FIM mid-training before the same post-training restores LiveCodeBench by +11.10, OJBench by +1.94 (within 0.46 of the Instruct ceiling), and FullStackBench-EN by +0.53. The six-benchmark average rises from 16.04 to 19.56 (+3.52 over post-training only), while the SWE-Bench-Verified gain on the same base is preserved (Table 1). Mid-training therefore improves the cost–benefit profile of agentic post-training: in-domain target metrics improve and the bulk of off-distribution erosion is undone in the same training run.

Cross-domain transfer to non-coding tool use. τ -bench and BFCL contain no Python code-editing data, and our mid-training corpus contains no tool-use trajectories. Mid-training nevertheless improves τ -bench by +3.90 and BFCL by +2.40 over post-training alone, with a directionally consistent recovery on Terminal-Bench (+1.25). Because the mid-training data carries no tool-use signal, the only mechanism by which it can affect these benchmarks is by installing a structural prior at mid-training time that survives subsequent post-training—direct evidence for the function-call/tool-call isomorphism (Section 2.1).

3.4 Ablation Studies

We run two controlled ablations on Qwen2.5-Coder-7B-Instruct with R2E-Gym post-training and a shared mid-training-free baseline. The first isolates the role of the chain-of-thought rationale; the second isolates the role of the function-aware selection pipeline. All selection variants in block (B) operate under a controlled budget—each variant selects exactly 200K single functions, with hard filters ($\text{LoC} \in [10, 200]$, dunder methods excluded) applied uniformly—so differences across rows reflect *which* functions are selected rather than *how many*. Block (C) varies mask granularity. Due to compute constraints, ablations run at 7B only.

FIM structure contributes independently of CoT distillation. Block (A) addresses a natural concern that improvements stem mainly from distilling reasoning. Removing the rationale entirely (*no CoT*) lifts the average by only +0.13, with +0.60 on Verified—a small effect consistent with the hypothesis that much of the underlying code is already familiar to the base model from pretraining, leaving little headroom for an unguided FIM signal alone. The more informative comparison is against a frontier teacher. Replacing Gemini-3 with rationales generated by the model under training (*self-CoT*) reaches 14.85, recovering 1.68 of the 2.30-point gap to *Gemini-3 CoT*: high-quality external rationales help, but a sizable fraction of the gain is reproducible without a frontier teacher. The function-aware FIM objective is therefore effective even when paired with the model’s own rationales, and our recipe does not merely distill from a frontier teacher.

Function-selection algorithm is the dominant lever. Block (B) varies the selection algorithm with CoT and budget held fixed. *Random* masking establishes the floor at 13.95, a margin of +0.78 over no mid-training: even random function-level masking on top of the basic hard filters helps modestly.

Table 3: Ablations on the 7B model with R2E-Gym post-training. (A) rationale source. (B) function-selection algorithm. (C) mask granularity (single vs. multi-function groups). All blocks share the baseline (*w/o mid-train*) and a controlled 200K-target budget; CoT is fixed to Gemini-3-Flash in (B) and (C). Bold marks the best configuration per block; the 80%/15%/5% mixture in (C) is the recipe used in our main results (Table 1), which is trained on the full corpus rather than the 200K budget here.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
<i>(A) Chain-of-thought source (selection fixed to “full”)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
+ FIM, no CoT	15.60	11.00	13.30
+ FIM, self-CoT (Qwen2.5-Coder-7B-Instruct)	16.40	13.30	14.85
+ FIM, Gemini-3-Flash CoT	17.00	14.20	15.60
<i>(B) Function-selection algorithm (CoT fixed to Gemini-3-Flash)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
Random	15.30	12.60	13.95
Gemini-selected	16.40	13.70	15.05
PDG only	16.10	13.60	14.85
PDG + complexity ($\hat{H}$)	16.50	13.60	15.05
PDG + inferability ($\hat{I}$)	16.70	14.00	15.35
Full (PDG + $\hat{H}$ + $\hat{I}$)	17.00	14.20	15.60
<i>(C) Mask granularity (selection fixed to “full”, CoT fixed to Gemini-3-Flash)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
Single-function only	17.00	14.20	15.60
85% single + 15% pair ($k = 2$)	17.20	14.60	15.90
95% single + 5% triple ($k = 3$)	17.00	14.40	15.70
80% single + 15% pair + 5% triple	17.40	14.80	16.10

Letting Gemini-3 choose which functions to mask (*Gemini-selected*) reaches 15.05 (+1.10 over Random), showing that frontier judgment on *which* function to mask is helpful but not by itself sufficient. Restricting the candidate pool to functions with at least one PDG neighbor (*PDG only*) reaches 14.85, between Random and Gemini-selected: structural connectivity alone contributes a small gain. Adding a scoring function on top of the PDG pool yields the next layer: *PDG* + $\hat{H}$ reaches 15.05 and *PDG* + $\hat{I}$ reaches 15.35, showing that intrinsic difficulty and contextual recoverability each contribute beyond the structural filter. Combining both via the harmonic-mean-like form (*Full*) reaches 15.60 (+0.25 over the better single-score variant), so $\hat{H}$ and $\hat{I}$ measure complementary properties and their combination is not redundant.

Mask granularity: pair masking helps, triples saturate. Block (C) mixes single-function targets with multi-function groups (Section 2.3.5). Adding 15% pair targets on top of single functions raises the average from 15.60 to 15.90 (+0.30); substituting 5% triple targets instead is essentially neutral (15.70). The full 80%/15%/5% mix achieves 16.10 (+0.50 over single-only), with the largest gain on Lite (+0.60). The pattern is consistent with the analysis in Section 4.2: training on cross-function dependencies disproportionately helps tasks whose gold patches span multiple functions, but the marginal return from larger groups diminishes as coupling becomes harder to maintain in the joint-masked context.

4 Analysis

The previous section established that FIM mid-training improves end-task metrics. We now ask *how* the resulting agent behaves differently and *where* along the trajectory the gains accrue. Both analyses use the 14B configuration with R2E-Gym, comparing the post-training-only baseline (*R2E-Gym*) against our recipe (*FIM-Midtrain* + *R2E-Gym*); statistics are run-means over three evaluation runs on SWE-Bench-Verified. The Lite analogue, the full action-type distribution, the no-patch mechanism, and a concrete trajectory contrast are deferred to Appendix D.

4.1 Recovery from Negative Observations

A trajectory contains a *negative observation* if any tool output matches a fixed set of error patterns (stack traces, “No replacement was performed”, shell errors, etc.; full list in Appendix B). The fraction of such trajectories is essentially identical across checkpoints (88.8% baseline vs. 91.8% ours), so the agents see comparable amounts of negative feedback. The *recovery rate*—the fraction of error-containing trajectories that nonetheless terminate with a passing patch—rises from 24.8% to 28.8% (+4.0 pp; +16% relative; Table 4). This is precisely the capability our framing predicts mid-training should support: continuing productively after an external return contradicts the model’s prior expectation. Mid-training also shifts the agent toward an iterate-and-verify policy, increasing edits per solved task from 3.3 to 7.4 and trajectory length from 15.1 to 23.6 steps; action-type breakdowns are in Appendix D.

Table 4: Headline trajectory metrics on SWE-Bench-Verified (14B, R2E-Gym), run-means over three runs. Full metrics in Appendix D.

Setting	Recovery rate (%)	Edits / solved	Steps / solved	Pass (%)
+ R2E-Gym	24.8	3.3	15.1	26.2
+ FIM-Midtrain + R2E-Gym	28.8	7.4	23.6	29.2

4.2 The Gain Concentrates on Multi-Function Reasoning

The most direct test of the isomorphism argument (Section 2.1) is whether mid-training preferentially helps tasks that require reasoning about cross-function dependencies inside a file. We stratify the 500 Verified tasks by the shape of the gold patch (Figure 4, Appendix D). On the 88 tasks whose gold patch modifies ≥ 2 functions within a single file, the baseline solves 13.6% on average and ours solves 22.7%, an absolute gain of +9.1 pp—more than $4\times$ the gain on the 341 single-function tasks (+2.1 pp). Per-instance head-to-head on this bucket shows ours uniquely solves about twice as many tasks as the baseline. Multi-file tasks ($n=71$) are not differentially helped, which we attribute to a granularity mismatch: our FIM operates within files (Appendix D). The slice where mid-training helps most is precisely the slice where the agent must follow control- and data-dependencies between caller-callee or sibling functions inside a file—the same structure exposed by function-aware FIM masking. A complementary outcome-distribution breakdown (Figure 5) shows the +15-task gain is driven by an order-of-magnitude reduction in *no-patch* failures alongside small reductions in localization errors; the FIM signal conditions the model to produce a non-empty span between prefix and suffix—a disposition that survives subsequent post-training.

5 Related Work

Mid-training and continued pretraining. A growing line of work identifies a distinct stage between large-scale pretraining and task-specific post-training, in which a model is further trained on a smaller, carefully curated corpus to install inductive biases that are hard to acquire from generic web text or downstream fine-tuning. MiniCPM [6] and OLMo [4] schedule such a stage to upweight high-quality or domain-relevant data near the end of pretraining, and Code-Llama [17] long-context adaptation can be read as a mid-training stage targeting context-length generalization. We adopt the same staging philosophy but apply it to a different end: installing an *agent-oriented* structural prior through function-aware FIM.

Fill-in-the-middle objectives. The fill-in-the-middle objective [1] is a standard ingredient in modern code LLM pretraining, with random-span variants used in Code-Llama [17], StarCoder [11], Qwen-Coder [7], and DeepSeek-Coder [5]. Our recipe extends and specializes prior FIM along three axes. First, masking targets are chosen at *function granularity* via program dependency graph analysis and a complexity–inferability double criterion, so the masked region corresponds to a unit of reasoning rather than an arbitrary span. Second, we embed an *explicit chain-of-thought* inside the FIM middle, supervising the model to produce a rationale before the code. Third, the objective lives in a *dedicated mid-training stage* rather than dissolved into pretraining.

Coding agent foundation models. Progress on coding agents has been driven by benchmarks such as SWE-Bench [10], scaffolds such as SWE-agent [22] and OpenHands [19], and trajectory-centric post-training pipelines such as SWE-Gym [14], R2E-Gym [9], and SWE-Smith [23]. Our work is

complementary: we use R2E-Gym and SWE-Smith unmodified on top of a mid-trained checkpoint, and the novelty lies one stage earlier—extracting a self-supervised signal from the structure already present in source code, which both raises in-domain agent metrics and mitigates the off-domain capability erosion that trajectory-only post-training otherwise inflicts.

Distillation from frontier models. Our default recipe uses Gemini-3-Flash to generate the chain-of-thought rationales embedded in each FIM middle span, placing it within the tradition of distilling capability from a strong teacher [13, 20, 3]. We isolate the contribution of FIM structure from CoT distillation through a controlled ablation in Section 3.4: removing the rationale still beats the no-mid-training baseline, and replacing the frontier teacher with self-generated rationales recovers a sizeable fraction of the gain, so the recipe is not distillation-bound.

6 Limitations and Discussion

We close by stating four limitations that scope our claims and flag directions for follow-up work. (i) *Python-only corpus and evaluation.* The mid-training corpus and the in-domain agent benchmarks (SWE-Bench-Verified, SWE-Bench-Lite) are exclusively Python; cross-language evidence comes only indirectly through FullStackBench-EN (Section 3.3), and whether the recipe transfers to Java, C++, or Rust ecosystems is left to future work. (ii) *Teacher dependency for CoT.* The default recipe relies on Gemini-3-Flash to synthesize rationales; the CoT-source ablation (Section 3.4) shows self-generated rationales recover most of the gain, but a fully open-source replication of our best configuration requires a comparably strong open teacher or acceptance of the residual gap. (iii) *Partial cross-base validation.* Our cross-base evidence comes from a single non-Qwen2.5-Coder configuration (Qwen3-8B with SWE-Lego), which simultaneously varies the post-training pipeline; the result should be read as “not specific to a single pretraining/post-training combination” rather than as a guarantee across all base-model families. (iv) *Modularity assumption.* Function-aware FIM presupposes meaningfully modular code; on monolithic scripts, generated code, or notebooks, the selection pipeline has fewer eligible targets and degrades to a smaller usable subset, a regime we do not study systematically here.

7 Conclusion

We started from the observation that a single step of a coding agent and a single function call site share the same four-part structure—*context*, *action*, *externally produced return*, *continuation*—making source code an internet-scale supply of agent-relevant signal. We turn this into function-aware FIM mid-training: a self-supervised stage that masks targets selected via program dependency graph analysis and a complexity–inferability double criterion, with chain-of-thought rationales embedded inside the FIM middle. Across three axes of robustness—model size (7B/14B), post-training pipeline (R2E-Gym, SWE-Smith, SWE-Lego), and base-model family (Qwen2.5-Coder, Qwen3)—the recipe delivers consistent gains on coding-agent benchmarks, and the same Python-only corpus also transfers to non-coding tool-use benchmarks (τ -bench, BFCL) where it has no a priori reason to help. Future work includes extending function-aware selection to non-Python languages and composing mid-training with reinforcement-learning post-training.

References

- [1] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle, 2022.
- [2] ByteDance Seed Foundation Code Team. FullStack Bench: Evaluating LLMs as full stack coders, 2024.
- [3] Sanchit Gandhi, Patrick von Platen, and Alexander M. Rush. Distil-Whisper: Robust knowledge distillation via large-scale pseudo labelling, 2023.
- [4] Dirk Groeneveld, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Raghavi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, Tushar Khot, William Merrill, Jacob Morrison, Niklas Muennighoff,

- 386 Aakanksha Naik, Crystal Nam, Matthew E. Peters, Valentina Pyatkin, Abhilasha Ravichander,
387 Dustin Schwenk, Saurabh Shah, Will Smith, Emma Strubell, Nishant Subramani, Mitchell
388 Wortsman, Pradeep Dasigi, Nathan Lambert, Kyle Richardson, Luke Zettlemoyer, Jesse Dodge,
389 Kyle Lo, Luca Soldaini, Noah A. Smith, and Hannaneh Hajishirzi. OLMo: Accelerating the
390 science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for
391 Computational Linguistics (ACL)*, 2024.
- 392 [5] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen,
393 Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. DeepSeek-Coder:
394 When the large language model meets programming – the rise of code intelligence, 2024.
- 395 [6] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei
396 Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi
397 Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia,
398 Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the potential
399 of small language models with scalable training strategies, 2024.
- 400 [7] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun
401 Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei
402 Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng
403 Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report, 2024.
- 404 [8] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang,
405 Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contami-
406 nation free evaluation of large language models for code, 2024.
- 407 [9] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-
408 Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents,
409 2025.
- 410 [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
411 Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In
412 *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- 413 [11] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao
414 Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii,
415 Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João
416 Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee,
417 Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang,
418 Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan
419 Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha
420 Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav
421 Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank
422 Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish
423 Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Fer-
424 randis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries.
425 StarCoder: May the source be with you!, 2023.
- 426 [12] Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in
427 command line interfaces, 2026.
- 428 [13] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and
429 Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of GPT-4,
430 2023.
- 431 [14] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe
432 Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.
- 433 [15] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica,
434 and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool
435 use to agentic evaluation of large language models. In *Proceedings of the 42nd International
436 Conference on Machine Learning (ICML)*, 2025.

- [16] Qwen Team. Qwen3 technical report, 2025.
- [17] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open foundation models for code, 2023.
- [18] SWE-Lego Team. SWE-Lego: Pushing the limits of supervised fine-tuning for software issue resolving, 2026.
- [19] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Yuan, Hao Pan, Yueqi Yang, Jiayi Hu, Mingyang Tan, Yongliang Liu, Yujia Liu, Tianbao Du, Hao Zhang, Yan Zhang, Jiawei Yu, Wanjuan Zhao, Robert M. Yang, Jingbo Yi, Mingzhang Zhang, Yuxiang Wang, Jiawei Wei, Theo Olausson, Tianyu Liu, Ren-Biao Sun, Wendong Liu, Tianyang Yang, Frank Mao, Yu Chen Wang, Wei Liu, Bill Lin, Boyuan Wei, Hanzhe Liang, Yifan Yan, Yibo Sun, Yuhao Sun, Hangrui Liu, Heran Wang, Hao Yan, Tao Tang, Yufeng Yang, Vijay Reddy, Saatvik Gandhi, Pravir Singh, Yu Yu, Tianhao Liu, Jianan Pang, Yumeng Wang, Saurabh Geng, Yang Hu, Lily Zhao, Yipeng Zhao, Pinjia Liu, Junda Bian, Junyi Mu, Junyu Tan, Hao Wang, Wangchunshu Lai, Shivansh Garg, Yunzhu Wang, Robert Lo, Junda Tao, Junyu Liu, Yifei Lyu, Yueting Wang, Mingyu Hu, Boqin Yang, Bo Liu, Hao Sun, Yangruibo Wang, John Yang, Mohit Bansal, Rada Mihalcea, Yang Cao, Edward Liu, Tongshuang Du, Ofir Press, Percy Liang, Justin Heinz, Vincent J. Hellendoorn, Marcel Boehme, Lianmin Zheng, Ion Stoica, Junfeng Wang, Peter Henderson, Prateek Goyal, Eric Mitchell, Heng Ji, Charlie Frank, Dan Hendrycks, Yiming Liang, Maosong Sun, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [20] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [21] Zhexu Wang, Yiwen Yang, Pengfei Zhu, Chuanrui Zheng, Jianxin Hu, Wangyue Huang, Yuchen Wu, Weizhe Liu, Shilong Wang, Yixuan Yan, Yiwen Liu, Yufei Sui, Lewei Zhang, Tianyu Yang, Mingyang Yu, Junjie Zhao, Weiqian Zhuang, Ke Lu, Junjie Yang, Cong He, Jian-Guang Lou, Zhouhan Lin, Junran Liu, Yidan Tao, and Tianhe Hua. OJBench: A competition level code benchmark for large language models, 2025.
- [22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [23] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025.
- [24] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

A Corpus Details

This appendix expands on the data-collection summary (Section 2.2). We report category coverage, per-property statistics, and the license inventory of the 968 released repositories.

A.1 License Inventory

Table 6 summarizes the license distribution across the 968 released repositories. The corpus is dominated by permissive licenses (MIT, Apache-2.0, BSD-2/3, ISC, Boost, MIT-0, Unlicense,

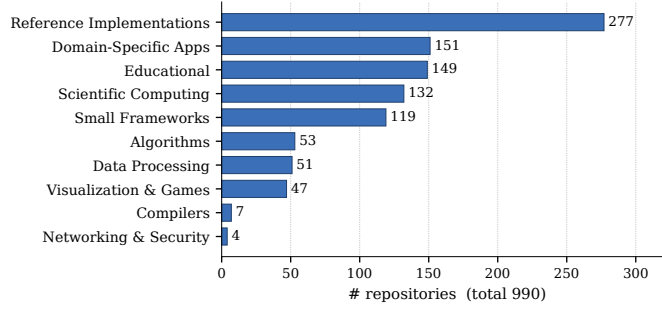


Figure 3: Distribution of the 968 source repositories across ten topic categories. The corpus is dominated by reference implementations, scientific computing, and small frameworks; compiler and networking/security tails are kept by design to maintain coverage diversity.

Table 5: Mid-training corpus statistics after decontamination and quality filtering. Token counts use the Qwen2.5-Coder tokenizer. All numeric quantities reflect the released corpus.

Property	Value
Source GitHub repositories	968
Topic categories	10
Filtered self-contained Python files	$\approx 78\text{K}$
Total lines of Python code	$\approx 33\text{M}$
Mean LoC per file	≈ 428
Mid-training token budget	$\approx 2.6\text{B}$
Single-function FIM targets	$\approx 320\text{K}$ ($\approx 2.0\text{B}$ tokens)
Multi-function targets ($k=2$)	$\approx 60\text{K}$ ($\approx 0.4\text{B}$ tokens)
Multi-function targets ($k=3$)	$\approx 20\text{K}$ ($\approx 0.2\text{B}$ tokens)
Mean target LoC	≈ 34
Targets with Gemini-3 CoT	100%
SWE-Bench source-repo overlap	0
Commit cutoff	before earliest SWE-Bench base

485 WTFPL), followed by copyleft (GPL-2/3, AGPL-3, LGPL-2.1/3, MPL-2.0, BSD-3 Clear) and
 486 Creative Commons licenses; per-license shares are **TBD** pending final tabulation. We will release the
 487 per-repository license alongside the repository list.

Table 6: License inventory of the 968-repository corpus. Counts are exact at release time; a small number of repositories with no declared or unidentifiable license are reported separately.

License	Count	License	Count
MIT License	TBD	GNU AGPL v3.0	TBD
Apache License 2.0	TBD	CC BY-SA 4.0	TBD
BSD 3-Clause	TBD	CC BY 4.0	TBD
BSD 2-Clause	TBD	CC0 1.0	TBD
GNU GPL v3.0	TBD	MPL 2.0	TBD
GNU GPL v2.0	TBD	GNU LGPL v2.1	TBD
ISC License	TBD	GNU LGPL v3.0	TBD
MIT-0	TBD	CC BY-NC 4.0	TBD
The Unlicense	TBD	BSD 3-Clause Clear	TBD
Boost Software License 1.0	TBD	Other / unidentifiable	TBD
Total: 968			

488 B Algorithmic Details

489 This appendix collects the full algorithmic details summarized in Section 2.3.

490 B.1 Single-Function FIM Target Selection

491 Algorithm 1 summarizes the per-file pipeline that produces single-function FIM targets. It composes
 492 the program dependency graph (Section 2.3.1), the complexity score $\hat{H}$ (Section 2.3.2), the inferability
 493 score $\hat{I}$ (Section 2.3.3), the difficulty penalty ρ (Appendix B.5), and the hard filters described below.

Algorithm 1 Single-function FIM target selection for one file.

Require: Source file s ; thresholds $\tau_d, \tau_{\text{FIM}}, \tau_{\hat{H}}$

```

1:  $T \leftarrow \text{ParseAST}(s)$ 
2:  $\mathcal{V}, \mathcal{E}_{\text{call}}, \mathcal{E}_{\text{sib}} \leftarrow \text{BuildPDG}(T)$  ▷ Section 2.3.1
3:  $\mathcal{C} \leftarrow \emptyset$ 
4: for each  $v \in \mathcal{V}$  do
5:   if  $v$  is a dunder method or  $\text{LoC}(v) \notin [10, 200]$  then
6:     continue
7:   end if
8:   Compute  $\hat{H}(v)$  and  $\hat{I}(v)$  ▷ Eqs. (1), (2)
9:   if  $\hat{H}(v) < \tau_{\hat{H}}$  then continue
10:  end if
11:   $\Delta(v) \leftarrow \max(0, \hat{H}(v) - \hat{I}(v)) / (\hat{H}(v) + \epsilon)$ 
12:   $\text{FIM}(v) \leftarrow \frac{\hat{H}(v) \hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v))$ 
13:  if  $\text{FIM}(v) \geq \tau_{\text{FIM}}$  then
14:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{v\}$ 
15:  end if
16: end for
17: return  $\mathcal{C}$  ▷ ranked by FIM descending

```

494 B.2 Program Dependency Graph: Call Resolution

495 For every `Call` node inside the body of $u \in \mathcal{V}$, we attempt to resolve its callee to some $v \in \mathcal{V}$, adding
 496 a directed edge $u \rightarrow v$ in $\mathcal{E}_{\text{call}}$. Resolution handles three Python idioms:

- 497 • **Direct module-level invocations:** `foo(...)` is resolved by qualified-name lookup in the
 498 same module.
- 499 • **Class instantiation:** `ClassName(...)` is resolved to `ClassName.__init__`.
- 500 • **Method calls:** `self.m(...)` or `cls.m(...)` inside class bodies is resolved within the
 501 enclosing class scope.

502 When a call cannot be resolved exactly we fall back to short-name matching against the qualified-
 503 name index, which recovers most cross-module references while admitting a controlled false-positive
 504 rate. Sibling edges $\mathcal{E}_{\text{sib}}$ are constructed by enumerating all pairs of methods belonging to the same
 505 class.

506 B.3 Complexity Score: Caps and Weights

507 The complexity score $\hat{H}(v)$ defined in Eq. (1) uses caps $(c_\ell, c_c, c_d) = (50, 10, 5)$ and weights
 508 $(w_\ell, w_c, w_d) = (0.4, 0.4, 0.2)$. The cap $c_\ell = 50$ on lines of code matches the median target length in
 509 our corpus; the cap $c_c = 10$ on McCabe complexity is roughly twice the median over our filtered
 510 functions; the cap $c_d = 5$ on nesting depth covers the tail of practical Python code. The cap value of
 511 2 in $\phi(x, c) = \min(x/c, 2)$ allows genuinely complex functions to score above the median without
 512 letting outliers dominate.

513 B.4 Inferability Score: Component Formulas

514 The five components of $\hat{I}(v)$ (Eq. 2) are defined below. Default mixing weights are $(\alpha, \beta, \gamma, \delta, \epsilon) =$
 515 $(0.30, 0.25, 0.20, 0.10, 0.15)$.

516 **Caller signal** C_{caller} . For each caller u of v , we score the specificity of its call sites by inspecting the
 517 syntactic form of each argument: literal constants are maximally informative, named variables less
 518 so, keyword arguments contribute proportionally, and a base score is awarded for the call site itself.
 519 The sum is normalized by an expected-callers constant.

520 **Callee signal** C_{callee} . The number of intra-file functions that v itself calls, normalized by a small
 521 expected-fan-out constant. A function that orchestrates known helpers is more recoverable than one
 522 calling only opaque externals.

523 **Signature signal** C_{sig} . A monotone aggregate of return-type annotation, parameter-type annotations,
 524 the descriptiveness of the function name (decomposed into snake-case parts), and the count of
 525 non-self parameters.

526 **Documentation signal** C_{doc} . Indicator of docstring presence. We do not score docstring informative-
 527 ness, both for robustness and because Section 2.4 generates a richer textual rationale.

528 **Class signal** C_{class} . For methods, the number of in-class siblings (normalized) plus a small bonus
 529 when an `__init__` is present in the same class.

530 B.5 Difficulty Δ , Penalty ρ , and Hard Filters

531 The penalty $\rho(\Delta(v))$ in Eq. (3) is built from the residual difficulty $\Delta(v)$, defined as the share of
 532 complexity that is not explained by context:

$$\Delta(v) = \frac{\max(0, \hat{H}(v) - \hat{I}(v))}{\hat{H}(v) + \epsilon} \in [0, 1). \quad (4)$$

533 The penalty ρ is one-sided: targets with $\Delta(v) \leq \tau_d$ are left intact, while targets above the threshold
 534 are damped by a Gaussian factor:

$$\rho(\Delta) = \begin{cases} 1, & \Delta \leq \tau_d, \\ \exp\left(-\frac{(\Delta - \tau_d)^2}{2\sigma^2}\right), & \Delta > \tau_d. \end{cases} \quad (5)$$

535 We use $\tau_d = 0.5$ and $\sigma = 0.2$. The asymmetry reflects an asymmetry in the underlying objective:
 536 trivially easy targets are already eliminated by the hard filters below, so we do not need to addition-
 537 ally reward high inferability; conversely, targets that remain hard even with full context become
 538 unlearnable noise and must be down-weighted.

539 **Hard filters.** Independent of the smooth score, we discard (i) functions with $\text{LoC}(v) \notin [10, 200]$ to
 540 control training-instance length; (ii) dunder methods (`__init__`, `__repr__`, etc.), which serve as
 541 scaffolding rather than reasoning targets; (iii) functions with $\hat{H}(v) < 0.15$ to remove trivial cases;
 542 and (iv) functions with $\text{FIM}(v) < 0.08$. A file with no surviving target contributes nothing to the
 543 corpus.

544 B.6 Worked Example: Scoring `Calculator.total`

545 To make the numbers in Figure 2(b) reproducible, this subsection walks through every quantity that
 546 goes into $\hat{H}$, $\hat{I}$, and the final FIM score for the running example. The target function is:

```

def total(self) -> int:
    """Sum positive integer entries."""
    s, n = 0, 0
    for v in self.history:
        if is_int(v):
            if v > 0:
                s = add(s, v)
                n += 1
    if n == 0: return 0
    return s

```

548 **Lines of code (LoC = 10).** Following Python’s `ast` convention, $\text{LoC}(v) = \text{end_lineno} - (\text{lineno} - 1)$, the number of source lines from the `def` keyword through the last statement, both inclusive.
 549 Counting the ten lines above (including the docstring) gives $\text{LoC} = 10$.
 550

551 **Cyclomatic complexity (CC = 5).** The McCabe counter starts at 1 and adds 1 per branching node:
 552 If/IfExp, For/While, ExceptHandler, (values-1) per BoolOp, and one per generator inside a
 553 comprehension. For Calculator.total: base (+1), the for loop (+1), if is_int (+1), if v>0
 554 (+1), if n==0 (+1), totalling CC = 5. Tuple unpacking and n+=1 are not branches.

555 **Maximum nesting depth (D = 3).** Depth increments only on structural containers (If, For, While,
 556 AsyncFor, With, AsyncWith, Try). The deepest path in total is for → if is_int → if v>0,
 557 giving D = 3.

558 **Complexity score $\hat{H} = 0.40$.** Plugging into Eq. 1 with the default $(w_\ell, w_c, w_d) = (0.4, 0.4, 0.2)$
 559 and $(c_\ell, c_c, c_d) = (50, 10, 5)$:

$$\begin{aligned} w_\ell \phi(10, 50) &= 0.4 \cdot 0.20 = 0.08, & w_c \phi(5, 10) &= 0.4 \cdot 0.50 = 0.20, \\ w_d \phi(3, 5) &= 0.2 \cdot 0.60 = 0.12, & \hat{H}(\text{total}) &= 0.08 + 0.20 + 0.12 = 0.40. \end{aligned}$$

560 **Inferability score $\hat{I} = 0.48$.** (i) *Caller specificity.* The only in-file caller is Calculator.mean,
 561 which invokes self.total() with no arguments. The specificity routine assigns 0.50 for the call
 562 site, 0 for arguments and keywords, and a +0.10 found bonus, yielding sp = 0.60. Aggregating over
 563 callers and normalizing by 3, $C_{\text{caller}} = \min(0.60/3, 1.0) = 0.20$ and $\alpha C_{\text{caller}} = 0.06$. (ii) *Callee*
 564 *count.* total calls is_int and add in-file (self.history.append(v) does not count because
 565 append is not in $\mathcal{V}$), so $C_{\text{callee}} = \min(2/4, 1.0) = 0.50$ and $\beta C_{\text{callee}} = 0.13$. (iii) *Signature.* Sub-
 566 bonuses +0.30 for → int, +0.20 for the descriptive name total, 0 for parameter type annotations,
 567 0 for non-self parameters: $C_{\text{sig}} = 0.50$ and $\gamma C_{\text{sig}} = 0.10$. (iv) *Documentation.* A docstring is
 568 present, so $C_{\text{doc}} = 0.5$ and $\delta C_{\text{doc}} = 0.05$. (v) *Class context.* total sits among three siblings; the
 569 base score is $\min(3/5, 1.0) = 0.60$, plus a +0.30 bonus for the distinct __init__ of Calculator,
 570 giving $C_{\text{class}} = 0.90$ and $\varepsilon C_{\text{class}} = 0.14$. Summing: $\hat{I}(\text{total}) = 0.06 + 0.13 + 0.10 + 0.05 + 0.14 =$
 571 0.48.

572 **Final FIM score and selection.** $\Delta(\text{total}) = \max(0, -0.08)/0.40 = 0$, so $\rho(\Delta) = 1$ and
 573 $\text{FIM}(\text{total}) = (0.40 \cdot 0.48)/(0.40 + 0.48) \approx 0.22 \geq \tau = 0.20$, selecting it as a mask target. The four
 574 short helpers are removed before scoring by the LoC hard filter; Calculator.__init__ is removed
 575 by the dunder filter.

576 B.7 Multi-Function Group Score

577 **Group score.** For a group $G = \{v_1, \dots, v_k\}$ enumerated from one of eight topology patterns,

$$\text{FIM}(G) = \text{Coup}(G) \cdot \frac{\hat{H}(G) \hat{I}(G)}{\hat{H}(G) + \hat{I}(G) + \epsilon} \cdot \rho(\Delta(G)), \quad (6)$$

578 where $\hat{H}(G) = \frac{1}{k} \sum_v \hat{H}(v)$ averages individual complexities, $\Delta(G)$ is defined analogously to the
 579 single-function case, and the coupling term $\text{Coup}(G) \in [0, 1]$ is a normalized count of intra-group
 580 edges plus a shared-state ratio:

$$\text{Coup}(G) = w_c \frac{|E_G^{\text{call}}|}{k(k-1)} + w_s \frac{|E_G^{\text{sib}}|}{\binom{k}{2}} + w_{\text{st}} \overline{\text{Jacc}}_G, \quad (7)$$

581 with $\overline{\text{Jacc}}_G$ the mean Jaccard similarity over pairs in G between sets of accessed instance attributes
 582 (self.x). We use $(w_c, w_s, w_{\text{st}}) = (0.50, 0.20, 0.30)$. The coupling term enters multiplicatively
 583 because, unlike the single-function case, the value of a multi-function target derives specifically
 584 from cross-function dependencies; a weakly coupled group is indistinguishable from k independent
 585 samples.

586 **Group inferability.** $\hat{I}(G)$ is recomputed under the assumption that *all* members of G are masked
 587 simultaneously: caller, callee, and sibling contributions from inside G are excluded, and docstrings of
 588 group members contribute zero (they are part of the masked body). Signature information is retained
 589 because we do not mask function signatures. This re-computation prevents groups from receiving
 590 spurious credit for intra-group references that disappear under joint masking.

591 **Topology taxonomy.** A candidate group must form a connected subgraph in $\mathcal{E}_{\text{call}} \cup \mathcal{E}_{\text{sib}}$. We
 592 enumerate eight patterns:

- 593 • $k = 2$: *caller-callee* (direct $A \rightarrow B$ edge); *co-callee* (two callees of a common caller);
 594 *sibling-coupled* (same-class methods sharing ≥ 1 instance attribute); *mutual-call* ($A \rightleftharpoons B$).
- 595 • $k = 3$: *call-chain* ($A \rightarrow B \rightarrow C$); *hub* ($A \rightarrow B, A \rightarrow C$); *fan-in* ($B \rightarrow A, C \rightarrow A$); *class-triad*
 596 (three pairwise state-sharing methods of the same class).

597 **Group selection.** We score every candidate group emitted by the topology enumerator and discard
 598 those with insufficient coupling, complexity, or score (defaults: $\text{Coup}(G) \geq 0.20$; $\hat{H}(G) \geq 0.18$;
 599 $\text{FIM}(G) \geq 0.10$). Groups exceeding LoC ratios of 0.30 ($k = 2$) or 0.40 ($k = 3$) of the file are
 600 rejected. Remaining candidates are sorted by $\text{FIM}(G)$ and selected greedily under non-overlap: a
 601 function may belong to at most one selected group per file. Per-file caps (5 pairs, 3 triples) prevent
 602 any single file from dominating the corpus.

Algorithm 2 Multi-function group FIM target selection for one file.

Require: Source file s ; per-function scores $\hat{H}(v), \hat{I}(v)$; thresholds $\tau_{\text{coup}}^G, \tau_{\hat{H}}^G, \tau_{\text{FIM}}^G$; per-file caps
 $(N_2, N_3) = (5, 3)$

- 1: $\mathcal{V}, \mathcal{E}_{\text{call}}, \mathcal{E}_{\text{sib}} \leftarrow \text{BuildPDG}(\text{ParseAST}(s))$
- 2: $\mathcal{G} \leftarrow \emptyset$
- 3: **for** each topology pattern $P \in \{\text{caller-callee, co-callee, sibling-coupled, mutual-call, call-chain, hub, fan-in, class-triad}\}$ **do**
- 4: Enumerate all connected subgraphs $G \subseteq \mathcal{V}$ matching P in $\mathcal{E}_{\text{call}} \cup \mathcal{E}_{\text{sib}}$
- 5: **for** each candidate G **do**
- 6: **if** $\text{LoC_ratio}(G, s) > \theta_k$ **or** $\text{Coup}(G) < \tau_{\text{coup}}^G$ **or** $\hat{H}(G) < \tau_{\hat{H}}^G$ **then**
- 7: **continue**
- 8: **end if**
- 9: Recompute $\hat{I}(G)$ under joint masking $\triangleright$ exclude intra-group references
- 10: $\Delta(G) \leftarrow \max(0, \hat{H}(G) - \hat{I}(G)) / (\hat{H}(G) + \epsilon)$
- 11: $\text{FIM}(G) \leftarrow \text{Coup}(G) \cdot \frac{\hat{H}(G) \hat{I}(G)}{\hat{H}(G) + \hat{I}(G) + \epsilon} \cdot \rho(\Delta(G))$
- 12: **if** $\text{FIM}(G) \geq \tau_{\text{FIM}}^G$ **then**
- 13: $\mathcal{G} \leftarrow \mathcal{G} \cup \{(G, P, \text{FIM}(G))\}$
- 14: **end if**
- 15: **end for**
- 16: **end for**
- 17: Sort $\mathcal{G}$ by $\text{FIM}(G)$ descending
- 18: $\mathcal{S} \leftarrow \emptyset$
- 19: **for** each $(G, P, \text{FIM}(G)) \in \mathcal{G}$ **do**
- 20: **if** any $v \in G$ already covered by $\mathcal{S}$ **or** per-pattern cap (N_2 for $k=2$, N_3 for $k=3$) exceeded **then**
- 21: **continue**
- 22: **end if**
- 23: $\mathcal{S} \leftarrow \mathcal{S} \cup \{G\}$
- 24: **end for**
- 25: **return** $\mathcal{S}$

603 **B.8 Worked Example: A caller-callee Pair**

604 To illustrate the group score, consider a small file with two top-level functions:

```

def normalize(text: str) -> str:
    """Lowercase and strip punctuation."""
    out = []
    for ch in text:
        if ch.isalnum() or ch.isspace():
            out.append(ch.lower())
    return "".join(out)

def word_freq(doc: str) -> dict[str, int]:
    """Count tokens of 'doc' after normalization."""
    cleaned = normalize(doc)
    counts = {}
    for tok in cleaned.split():
        counts[tok] = counts.get(tok, 0) + 1
    return counts

```

Topology and pre-checks. The pair $G = \{\text{normalize}, \text{word_freq}\}$ matches the *caller-callee* pattern with one call edge $\text{word_freq} \rightarrow \text{normalize}$. They are top-level (no class), so the sibling component of Coup is zero and the shared-state component is zero (no `self._` accesses). The combined LoC is roughly 14 lines, well under the $\theta_2 = 0.30$ file-ratio cap.

Coupling. With one call edge in a pair, $|E_G^{\text{call}}|/(k(k-1)) = 1/2$. Plugging into Eq. 7 with $(w_c, w_s, w_{st}) = (0.50, 0.20, 0.30)$: $\text{Coup}(G) = 0.50 \cdot 0.5 + 0 + 0 = 0.25$, comfortably above the $\tau_{\text{coup}}^G = 0.20$ floor.

Group $\hat{H}$. Per-function complexities (computed as in Section 2.3.2) come out to roughly $\hat{H}(\text{normalize}) \approx 0.36$ (LoC=7, CC=3, D=2) and $\hat{H}(\text{word_freq}) \approx 0.30$ (LoC=7, CC=2, D=2), giving $\hat{H}(G) \approx 0.33$.

Group $\hat{I}$ under joint masking. Both functions are masked, so the call edge between them no longer contributes to C_{caller} or C_{callee} . Signature information is retained: both have descriptive names and full type annotations (strong C_{sig}); both have docstrings (small but nonzero C_{doc}). With no surviving caller/callee/class signal, the group inferability collapses to $\hat{I}(G) \approx 0.18$, dominated by the signature term.

Final group FIM. The harmonic-mean-like product $\hat{H}\hat{I}/(\hat{H} + \hat{I}) \approx (0.33 \cdot 0.18)/0.51 \approx 0.116$, $\Delta(G) = (0.33 - 0.18)/0.33 \approx 0.45 < \tau_d = 0.5$ so $\rho = 1$, and $\text{Coup}(G) = 0.25$: $\text{FIM}(G) \approx 0.25 \times 0.116 \approx 0.029$. This example sits below the $\tau_{\text{FIM}}^G = 0.10$ floor and would therefore *not* be selected—the toy module is too small for the joint-masking inferability to remain large enough. Real corpus pairs that pass the threshold typically come from class-method pairs with non-trivial sibling and shared-state coupling, where $\hat{I}(G)$ retains substantial C_{class} contribution even under joint masking. The walkthrough nonetheless shows the mechanism: joint masking strictly reduces $\hat{I}$ relative to the single-function average, and the coupling factor must compensate.

B.9 Chain-of-Thought Prompt Template

The chain-of-thought rationale embedded inside each FIM middle span (Section 2.4) is generated by Gemini-3-Preview using the following prompt skeleton:

```

[System] You are an expert software engineer.
[User] Below is a Python source file with one function body redacted (shown as <MASKED>).
<file with target body redacted>
The redacted function has the following signature and (if present) docstring:
<signature + docstring>
The original function body was:
<original body>
Write a concise step-by-step rationale (in natural language) that explains how the body's
behavior could be recovered from the surrounding file—for example, by referencing call
sites, helper functions, and class-level state—without simply restating the code.

```

Rationales are constrained to at most 300 tokens and are filtered by length and basic format checks before training. We use greedy decoding; retry logic and quality filters will be released with the corpus.

B.10 Negative-Observation Patterns

The recovery-rate analysis (Section 4.1) flags a tool output as a *negative observation* when it matches any of the following patterns: Python stack-trace prefixes (Traceback (most recent call last):); error class names (SyntaxError, IndentationError, NameError, ImportError, AttributeError, TypeError, ValueError); shell error markers (command not found, permission denied, No such file or directory); and harness-specific failure strings (No replacement was performed, Patch did not apply, tests failed). Patterns are matched case-insensitively.

C Training Hyperparameters

We use LLAMAFACTORY for FIM mid-training, R2E-Gym post-training, and SWE-Lego post-training, and TORCHTUNE for SWE-Smith post-training. All runs use AdamW with bf16 mixed precision and a cosine learning-rate schedule. The LLAMAFACTORY runs use DeepSpeed ZeRO-3 sharding, FlashAttention-2, and the Liger kernel; the TORCHTUNE run uses fused AdamW, activation checkpointing, and torch.compile. For R2E-Gym and SWE-Smith we follow the official release scripts; for SWE-Lego we follow the official recipe except that we train for 2 epochs instead of 4 to prevent overfitting (all other hyperparameters are unchanged). Tables 7 and 8 list the exact settings; effective (global) batch sizes assume 8 GPUs.

Table 7: FIM mid-training hyperparameters (LLAMAFACTORY). The same recipe is applied to all three base models (Qwen2.5-Coder-7B-Instruct, Qwen2.5-Coder-14B-Instruct, and Qwen3-8B).

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1.0×10^{-5}
LR schedule	Cosine
Warmup ratio	0.1
Weight decay	0.05
Epochs	1
Per-device batch size	1
Gradient accumulation	16
Effective batch size	128 (8 GPUs $\times$ 1 $\times$ 16)
Sequence length	32,768
Precision	bf16
RoPE scaling	YaRN
System	DeepSpeed ZeRO-3, FlashAttention-2, Liger kernel

D Extended Behavioral Analysis (SWE-Bench-Verified)

This appendix collects the trajectory-level material that supplements the headline analysis in Section 4. Unless stated otherwise, statistics are run-means over three independent evaluation runs of each checkpoint on SWE-Bench-Verified (500 instances per run).

D.1 Pass Rate by Gold-Patch Shape

Figure 4 reports the pass-rate stratification used in Section 4.2: the largest gain (+9.1 pp) is on multi-function single-file tasks ($n=88$), more than $4\times$ the +2.1 pp gain on single-function tasks ($n=341$); multi-file tasks ($n=71$) remain hard for both checkpoints ($\sim 11.3\%$ each).

D.2 Full Trajectory Metrics

Table 9 reports the full trajectory metrics summarized in Table 4, including unsolved-task breakdowns and output/context summaries.

Table 8: Agentic post-training hyperparameters for the three pipelines. R2E-Gym and SWE-Smith follow their official released scripts; SWE-Lego follows the official recipe except for the epoch count (2 instead of 4, to prevent overfitting).

Hyperparameter	R2E-Gym	SWE-Smith	SWE-Lego
Framework	LlamaFactory	torch tune	LlamaFactory
Base model	Qwen2.5-Coder-7B-Inst.	Qwen2.5-Coder-7B-Inst.	Qwen2.5-72B (FIM mid-trained)
Optimizer	AdamW	AdamW (fused)	AdamW
Learning rate	1.0×10^{-5}	1.0×10^{-4}	1.0×10^{-4}
LR schedule	Cosine	Cosine	Cosine
Warmup	ratio 0.05	5 steps	ratio 0.1
Weight decay	0.0 (default)	0.01	0.01
Epochs	2	3	2 [†]
Per-device batch size	1	1	1
Gradient accumulation	1	4	8
Effective batch size	8	32	64
Sequence length	20,480	32,768	131,072
Precision	bf16	bf16	bf16
System	ZeRO-3 (offload), FA-2, Liger	FSDP, act. ckpt., compile	ZeRO-3, FA-2, Liger

[†]We deviate from the official SWE-Lego recipe (4 epochs) and use 2 epochs to prevent overfitting; all other hyperparameters follow the official scripts.

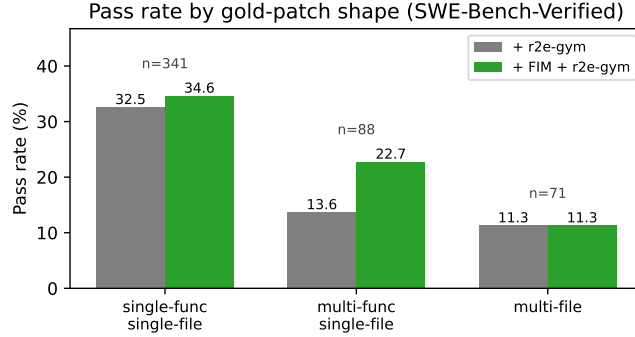


Figure 4: Pass rate on SWE-Bench-Verified (14B + R2E-Gym) stratified by gold-patch shape, run-means over three evaluation runs per checkpoint.

673 D.3 Iterate-and-Verify: Action-Type Distribution

674 Mid-training shifts the action-type distribution toward an iterate-and-verify policy. The share of
675 search actions falls from 15.3% to 11.0%, while the share of `execute_bash` actions rises from
676 19.5% to 24.6%: the agent spends a smaller fraction of its budget on passive repository lookup
677 and a larger fraction on actively running scripts and tests, then refining its edit. The cost is more
678 steps—ours hits the step cap on $\sim 45\%$ of trajectories vs. 7% for the baseline—but this cost is paid
679 on the unsolved tail; on the solved set the additional steps translate into more correct patches.

680 D.4 Failure-Mode Breakdown

681 We label every failed trajectory using a deterministic patch-quality cascade: *no-patch* (output diff is
682 empty); *localization error* (non-empty patch but zero file overlap with the gold patch); *patch error*
683 (file overlap with gold patch but tests fail). Figure 5 reports the outcome distribution: across the three
684 runs, mid-training cuts no-patch failures by an order of magnitude ($\sim 11 \rightarrow \sim 1$ trajectories per run),
685 trims localization errors slightly ($\sim 131 \rightarrow \sim 126$), and leaves the patch-error count essentially flat
686 ($\sim 227 \rightarrow \sim 227$). The +15-task gain in solved count is therefore primarily driven by the no-patch
687 mode collapse.

Table 9: Trajectory-level behavioral metrics on SWE-Bench-Verified (14B, R2E-Gym), run-means over three evaluation runs of each checkpoint. Steps, edits, and errors-encountered are means per task. “Empty patch” is the fraction of *failed* trajectories whose final `output_patch` is empty; “Loc. correct” is the fraction of all trajectories that locate at least one file overlapping the gold patch; “Ctx @ success” is the mean prompt context length on solved trajectories.

Setting	Steps / task		Edits / task		Errors enc.		Recovery rate (%)
	solved	unsolved	solved	unsolved	solved	unsolved	
+ R2E-Gym	15.1	22.4	3.3	5.4	3.5	5.8	24.8
+ FIM-Midtrain + R2E-Gym	23.6	32.7	7.4	10.9	6.2	8.1	28.8
<i>Output and context summaries</i>							
	Empty patch (% failed)		Loc. correct (% all)		Ctx @ success (tok)		Pass (%)
+ R2E-Gym	3.0		70.6		11,826		26.2
+ FIM-Midtrain + R2E-Gym	0.3		73.4		16,397		29.2

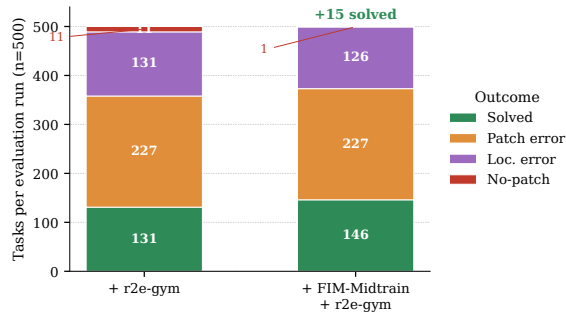


Figure 5: Outcome distribution per evaluation run on SWE-Bench-Verified (14B, R2E-Gym), averaged over three runs.

688 D.5 No-Patch Mode: Mechanism

689 The baseline no-patch failures recovered by mid-training are distributed across all task buckets and
690 are not concentrated in any single repository. In every recovered case the baseline emitted `<finish>`
691 without applying any `str_replace`, while ours applied at least one edit. We interpret this as a direct
692 consequence of the FIM training signal: under FIM the model is always conditioned to produce a
693 non-empty token span between the prefix and suffix, and this disposition survives the post-training
694 pipeline.

695 D.6 Multi-File Tasks Are Not Differentially Helped

696 On the 71 Verified tasks whose gold patch spans ≥ 2 files, both checkpoints solve $\sim 11.3\%$ on
697 average and per-instance head-to-head on this bucket is balanced. We attribute this to a granularity
698 mismatch: our function-aware FIM mid-training operates at the function level within files, so cross-
699 file coordination is not directly trained for. Extending the selection algorithm to cross-file function
700 pairs is a natural follow-up.

701 D.7 A Concrete Contrast

702 The shift in termination behavior is clearest on cases where the baseline quits prematurely. On
703 `scikit-learn-26323`, for instance, the baseline trajectory runs for a single step in every run,
704 immediately emits `<finish>` with an empty edit and the wrong target file, and is scored as a failure;
705 on the same task our agent runs ~ 41 steps, applies on the order of 20 `str_replace` operations,
706 observes multiple negative tool returns, and recovers to a passing patch. Pooling across the three
707 runs, on the order of ~ 16 of the ~ 55 tasks that ours uniquely solves on Verified have this signature:
708 the baseline emits `<finish>` with zero file overlap with the gold patch (often before any negative

observation has even arrived), while ours iterates past intermediate signals and converges on a correct edit. Mid-training does not enable a categorically new capability on these instances; it changes the agent’s stopping policy.

E Behavioral Analysis on SWE-Bench-Lite

This appendix replicates the Verified analysis on the SWE-Bench-Lite test split (300 instances). The same trajectory-parsing pipeline and failure-mode taxonomy are used; statistics are run-means over three independent evaluation runs of each checkpoint.

The qualitative trends from Verified hold on Lite: mid-training raises the recovery rate from 18.8% to 22.1%, eliminates the no-patch failure mode entirely (baseline averages ~ 8 no-patch failures per run, ours averages 0), and increases edit iteration on solved trajectories from 4.4 to 7.8 `str_replace` operations per task. The *multi-function* concentration of gain documented on Verified is not visible on Lite because Lite contains no multi-file tasks and only 54 multi-function single-file tasks; the bulk of Lite is single-function single-file tasks, on which the gain is +4.9 pp (19.5% $\rightarrow$ 24.4%).

Table 10: Trajectory-level behavioral metrics on SWE-Bench-Lite (14B, R2E-Gym), run-means over three evaluation runs of each checkpoint. Definitions match Table 9.

Setting	Steps / task		Edits / task		Errors enc.		Recovery rate (%)
	solved	unsolved	solved	unsolved	solved	unsolved	
+ R2E-Gym	17.5	21.3	4.4	5.2	4.8	5.4	18.8
+ FIM-Midtrain + R2E-Gym	24.6	31.7	7.8	11.1	5.7	7.5	22.1
<i>Output and context summaries</i>							
	Empty patch (% failed)		Loc. correct (% all)		Ctx @ success (tok)		Pass (%)
+ R2E-Gym	3.3		65.0		13,958		18.0
+ FIM-Midtrain + R2E-Gym	0.0		71.0		18,104		22.0

Failure-mode counts on Lite move in the same direction as on Verified: the no-patch mode is eliminated ($\sim 8 \rightarrow 0$ trajectories per run), localization errors fall by about 11, and patch errors rise by about 7, reflecting the same iterate-and-verify shift.

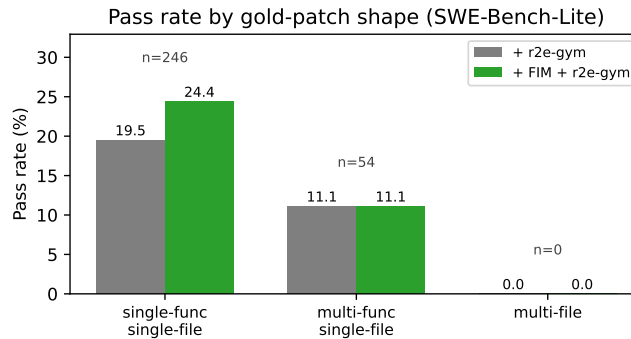


Figure 6: Pass rate on SWE-Bench-Lite stratified by gold-patch shape, averaged over three evaluation runs per checkpoint. Lite contains no multi-file tasks; the multi-function single-file bucket is small ($n=54$) and shows no gain on this slice, with the +4.0 pp end-task improvement coming entirely from the single-function single-file bucket ($n=246$, +4.9 pp).

Per-instance head-to-head. Aggregating per-task outcomes across the three runs (run-majority vote per checkpoint), the differential favors ours on Lite by a $\sim 1.8\times$ ratio overall and by a $\sim 2.1\times$ ratio on the single-function single-file bucket. On the multi-function single-file bucket the differential is balanced, consistent with the small absolute number of such tasks in Lite.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the function-call/agent-step correspondence, the function-aware FIM mid-training method, and the three-axis robustness claim, each of which is supported by the experiments in Section 3.2 (Table 1) and Section 3.3 (Table 2).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 explicitly enumerates four limitations: Python-only corpus and evaluation, teacher dependency for CoT generation, partial cross-base validation (only one non-Qwen2.5-Coder configuration), and the modularity assumption underlying function-aware selection.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is empirical and contains no theorems or formal theoretical results; the function-call/agent-step correspondence motivating the method is described as a structural analogy rather than a formal claim (Section 2.1).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 2 fully specifies the selection pipeline, scoring functions, and FIM format, with single- and multi-function selection algorithms, hyperparameters, and CoT prompt templates given in Appendix B and corpus statistics in Appendix A; Section 3.1 specifies models, post-training pipelines (R2E-Gym, SWE-Smith, SWE-Lego), evaluation protocol, and seeds.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We will release the 968-repository corpus and training code at camera-ready time; we do not include them with this submission to preserve double-blind anonymity. Algorithm pseudocode for both single- and multi-function selection is in Appendix B.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 3.1 describes the mid-training and post-training setup, benchmarks, evaluation protocol (means over three seeds), and agent harness; the full hyperparameter list and token budgets are in Appendix C, and selection-pipeline thresholds, weights, and filters are in Appendix B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Table 1 reports means over three independent evaluation seeds with $\pm$ standard-deviation bands on the main coding-agent benchmarks (SWE-Bench-Verified and SWE-Bench-Lite), as described in the evaluation protocol in Section 3.1.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: We use BF16 mixed-precision training and report token budgets and pipeline configurations in Section 3.1 and Appendix C, but aggregate GPU-hours and exact hardware breakdowns are not finalized at submission time and will be added in the camera-ready version.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work involves no human subjects, no PII, and uses only permissively-licensed open-source code (license inventory in Appendix A.1); benchmarks used (SWE-Bench, LiveCodeBench, OJBench, FullStackBench-EN, Terminal-Bench, τ -bench, BFCL) are publicly available academic resources.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper focuses on technical contributions and does not include a dedicated broader-impacts section; the work is incremental over existing code LLMs and agentic post-training pipelines, with positive impact from improved coding-agent reliability and dual-use risks comparable to those of the underlying base models, briefly touched on in Section 6.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are a fine-tuning corpus of permissively-licensed open-source Python code and code-completion/agent fine-tuned models that do not introduce generative capabilities for harmful content beyond what the underlying Qwen2.5-Coder/Qwen3 base models already provide; no scraped private data or high-misuse-risk content is released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All base models (Qwen2.5-Coder, Qwen3), post-training pipelines (R2E-Gym, SWE-Smith, SWE-Lego, OpenHands, SWE-agent), and benchmarks (SWE-Bench, LiveCodeBench, OJBench, FullStackBench-EN, Terminal-Bench, τ -bench, BFCL) are cited with their original publications, and the per-repository license inventory of the 968-repo corpus is in Appendix A.1.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: The 968-repository decontaminated corpus, selection pipeline, and mid-trained checkpoints are documented in Section 2.2 and Appendix A (corpus statistics, topic categories, licenses) but are not released with this submission to preserve double-blind anonymity; they will be released with full documentation at camera-ready time.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or research with human subjects; the corpus is constructed from publicly available GitHub repositories and chain-of-thought rationales are generated by an LLM (Gemini-3-Preview/Flash), not by human annotators.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Section 2.4 describes the use of Gemini-3-Flash to generate chain-of-thought rationales and Gemini-3-Flash to score (CoT, completion) pairs for filtering, with the full prompt templates provided in Appendix B.9; LLMs are also used as the trained models themselves (Qwen2.5-Coder, Qwen3) per Section 3.1.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.